EX18 - Advanced Functions - *args, **kwargs and Functions as Objects

Beginner Step-by-Step Guide

This guide follows the real file `ex18_advanced_functions.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Practice different ways of passing arguments to functions and learn that a function can itself be stored and passed as data.

Problem variables / elements

Name	Meaning
initial_energy	g*h0 + 0.5*v0**2
average_measurements	average of N values
apply_operation	executes a received function

Equation or logic on paper

```
energy_0 = g*h0 + 0.5*v0**2
average = sum(values)/len(values)
```

Manual check

For h0=100, v0=50, g=9.81: energy_0=981+1250=2231 J/kg. With g=1.62: 162+1250=1412 J/kg.

2. What we already know and what is new

Already seen:

- def
- default arguments
- dict
- tuple
- for

New in this exercise:

- positional vs keyword
- *args in a definition
- **kwargs in a definition
- * at call time
- ** at call time
- function as an object
- higher-order function

3. New Python concepts

1. *args in a definition

Syntax: `def average_measurements(*values):`

The * means a variable number of positional arguments may arrive. Inside they are packed into a tuple called values.

2. **kwargs in a definition

Syntax: `def simulation_summary(**kwargs):`

The ** means a variable number of keyword arguments may arrive. Inside they are packed into a dict.

3. * at call time

Syntax: `average_measurements(*measurements)`

Here * does the opposite: it unpacks a sequence into positional arguments.

4. ** at call time

Syntax: `initial_energy(**config)`

Here ** unpacks a dictionary into keyword arguments. Keys must match initial_energy explicit parameter names.

5. Function as an object

Syntax: `apply_operation(add, 5, 3)`

add has no parentheses because the function itself is being passed rather than executed at that point.

4. Code step by step

Step 1. Positional and keyword

```
initial_energy(h0, v0, g)
initial_energy(h0=1.0, v0=50.0, g=9.81)
```

The first call matches by order; the second by name.

Step 2. Use and override a default

```
def initial_energy(h0, v0, g=9.81): ...
initial_energy(h0, v0)
initial_energy(h0, v0, g=1.62)
```

If g is omitted, 9.81 is used. If supplied, the call value overrides the default for that call.

Step 3. Receive *args

```
def average_measurements(*values):
    average = sum(values) / len(values)
```

values is a tuple whether 2 or 5 arguments are supplied.

Step 4. Receive **kwargs

```
def simulation_summary(**kwargs):
    for key, value in kwargs.items():
        ...
```

kwargs is a dict. items() iterates over key-value pairs.

Step 5. Unpack a list

```
measurements = [10.2, 10.5, 9.9, 10.1]
average_measurements(*measurements)
```

Conceptually equivalent to average_measurements(10.2,10.5,9.9,10.1).

Step 6. Unpack a dict

```
config = {"h0":0.0, "v0":50.0, "g":9.81}
initial_energy(**config)
```

Important: initial_energy does not define **kwargs. **config converts keys into keyword arguments for normal parameters.

Step 7. Pass function without executing it

```
adding_values = apply_operation(add, 5, 3)
multiplying_values = apply_operation(multiply, 5, 3)
```

Inside apply_operation, operation(x,y) executes. operation refers to add or multiply depending on the call.

5. Execution flow

```
call styles
 -> packing with *args/**kwargs
 -> unpacking with *sequence/**dict
 -> pass function object
 -> execute it later
```

6. Expected result / check

apply_operation(add,5,3) returns 8; apply_operation(multiply,5,3) returns 15. average_measurements accepts different numbers of values.

7. Common beginner mistakes

- Confusing *args in a definition with *list at call time.
- Confusing **kwargs with **config at call time.
- Thinking initial_energy(**config) means initial_energy uses **kwargs.
- Passing add() when add is required.
- Putting positional arguments after keyword arguments.

8. If you remember only five things

- Positional matches by order; keyword by name.
- *args packs positional arguments into a tuple.
- **kwargs packs keyword arguments into a dict.
- * and ** at call time unpack structures.
- A function is passed without () when it should not execute yet.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.